

Trabajo Práctico — Semana 3

Optimización asistida por IA sobre Food Store: filtros, planes de ejecución e índices

Esta práctica aplica los contenidos de la Semana 3 sobre el proyecto integrador Food Store, con la inteligencia artificial como motor primario del trabajo, tal como lo establece la cátedra. El objetivo no es que la IA optimice sola, sino que el estudiante mida, decida y justifique cada cambio con evidencia de EXPLAIN ANALYZE — la IA propone, el estudiante verifica.

Objetivos de la práctica

- Analizar consultas lentas sobre una base Food Store poblada masivamente y optimizarlas midiendo la mejora real.
- Usar la IA (OpenCode / Kiro) como motor primario para proponer reescrituras e índices a partir de un plan de ejecución real.
- Contrastar la explicación de un plan generada por IA contra el plan real, detectando imprecisiones.
- Generar consultas resumen y subconsultas bajo especificación precisa, verificando la equivalencia de resultados contra una alternativa propia.
- Documentar cada decisión —aceptada o descartada— en la bitácora de uso de IA (DUIA).

Requisitos previos

- Base Food Store creada a partir de schema_completo.sql, con el seed de data.sql aplicado.
- Repositorio Git de la cátedra con OpenCode y Kiro configurados, según el protocolo de seguridad: copia, transacción, respaldo.
- Acceso a psql / DBeaver / pgAdmin para ejecutar EXPLAIN ANALYZE.

Parte 1 — Poblar la base masivamente con datos generados por IA

Antes de poder medir una optimización, hace falta un volumen de datos realista. Los seeds de la Guía Docente cargan apenas un puñado de filas; para que un Seq Scan sea lento hace falta escala.

1. Pedir a la IA (OpenCode) un script SQL o un generador (por ejemplo, con generate_series) que cargue: al menos 50.000 productos distribuidos en las categorías existentes, 20.000 usuarios y 200.000 pedidos con sus detalles.

2. Leer línea por línea el script generado antes de ejecutarlo: verificar que respeta las restricciones (CHECK, UNIQUE, FK) y que no toca la base de producción.
3. Ejecutarlo dentro de una transacción, sobre una copia de la base del proyecto, con respaldo previo — protocolo de seguridad de la cátedra.
4. Correr ANALYZE sobre las tablas afectadas para que el optimizador actualice sus estadísticas antes de medir.

```
-- Ejemplo de prompt / spec para la IA (a completar y ajustar por el estudiante):
-- "Generá un script SQL para PostgreSQL que inserte 50.000 filas en producto,
-- distribuidas de forma pareja entre las categorías existentes, con precios
-- entre 500 y 5000 y stock aleatorio entre 0 y 200. Usá generate_series y
-- no uses PL/pgSQL si no es necesario. No modifiques ninguna otra tabla."
```

Parte 2 — Laboratorio: consultas lentas, EXPLAIN y optimización medida

Con la base ya poblada masivamente, se eligen consultas reales de queries.sql que se vuelven lentas a esta escala — por ejemplo, el listado de productos por categoría o el historial de pedidos por usuario sin índice — y se optimizan siguiendo el flujo profesional de la cátedra: medir antes, proponer, medir después, decidir con datos.

2.1 Consigna

1. Elegir al menos tres consultas de queries.sql (*o variantes propias sobre el modelo*) que tarden de forma perceptible sobre la base masiva.
2. Para cada una, correr EXPLAIN ANALYZE y guardar el plan completo (*captura o texto*) antes de cualquier cambio.
3. Pasarle a la IA el plan real obtenido y pedirle que proponga reescrituras y/o índices que lo mejoren, justificando cada propuesta en términos del propio plan (qué nodo ataca, por qué esperaría que cambie).
4. Leer línea por línea cada CREATE INDEX o reescritura propuesta antes de aplicarla. Rechazar toda propuesta que no se entienda por completo.
5. Aplicar los cambios aceptados sobre la copia de trabajo y volver a medir con EXPLAIN ANALYZE.
6. Completar la tabla comparativa de la sección 2.2 para cada consulta.

2.2 Tabla de resultados (incluir en la entrega, una fila por consulta)

Consulta	Plan antes (nodo, cost, tiempo real)	Cambio aplicado	Plan después (nodo, cost, tiempo real)	Mejora
1	Nodo: Parallel Seq Scan on compra. Cost: 0.00..5598.71. Tiempo real: nodo 6.666 ms; global 47.202 ms.	CREATE INDEX idx_compra_situacion_dia ON compra(situacion, dia);	Nodo: Bitmap Heap Scan on compra (vía Bitmap Index Scan). Cost: 30.44..3515.00. Tiempo real: nodo 0.183 ms; global 6.768 ms.	De 47.202 ms a 6.768 ms → 7.0x
2	Nodo: Seq Scan on articulo a. Cost:	Reescritura de ILIKE '%...%' a	Nodo: Bitmap Heap Scan on articulo (vía Bitmap	De 38.587 ms a 0.077 ms →

	0.00..1395.12. Tiempo real: nodo 0.015..38.510 ms; global 38.587 ms.	to_tsvector('spanish', descripcion) @@ to_tsquery(...) y CREATE INDEX idx_articulo_desc ON articulo USING GIN (to_tsvector('spanish', descripcion));	Index Scan sobre el índice GIN). Cost: 14.15..600.56. Tiempo real: nodo 0.018..0.018 ms; global 0.077 ms.	501.1x
3	Nodo: Parallel Seq Scan + Sort (external merge Disk). Cost: 0.00..3449.53. Tiempo real: nodo 0.006..6.333 ms; global 139.668 ms.	CREATE INDEX idx_detalle_articulo ON detalle_compra(articulo_id);	Nodo: Index Scan using idx_detalle_articulo on detalle_compra dc. Cost: 0.29..5960.45. Tiempo real: nodo 0.026..32.713 ms; global 90.670 ms.	De 139.668 ms a 90.670 ms → 1.5x

Criterio de aceptación

Ninguna propuesta de la IA se aplica “porque lo dijo la IA”: se aplica solo si el estudiante puede explicar, en la defensa oral, por qué el índice o la reescritura deberían mejorar ese plan en particular, y la medición posterior lo confirma. Si una propuesta no mejora el tiempo real, se documenta igual —qué se esperaba, qué pasó, por qué— en lugar de descartarla en silencio.

Parte 3 — Lectura crítica de planes interpretados por IA

Un asistente de IA puede explicar en lenguaje natural qué hace un plan de EXPLAIN ANALYZE, pero esa explicación puede contener imprecisiones —por ejemplo, confundir un costo estimado con un tiempo real, o atribuir la mejora a la causa equivocada—. Esta parte entrena la lectura crítica de esas explicaciones.

1. Tomar uno de los planes reales obtenidos en la Parte 2 (con índice ya aplicado).
2. Pedirle a la IA que lo explique en lenguaje natural, nodo por nodo, sin mostrarle más contexto que el texto del plan.
3. Contrastar esa explicación contra el plan real, frase por frase, y marcar cualquier afirmación incorrecta o imprecisa (por ejemplo: “confunde cost con milisegundos”, “atribuye la mejora al índice equivocado”, “ignora el Rows Removed by Filter”).

Plan 1 Antes (Sin índice adecuado)

Nodo: Parallel Seq Scan on pedido

¿Qué hizo la base de datos? Como no había un índice que sirviera para filtrar rápido, PostgreSQL recurrió a un escaneo secuencial en paralelo: repartió el recorrido de la tabla pedido entre varios procesos trabajadores, leyendo las filas una tras otra desde el inicio hasta el final.

¿Por qué demoró tanto? Necesitó recorrer la totalidad de las filas de la tabla para evaluar, en cada una, dos filtros: el estado “terminado” y la antigüedad dentro de los últimos 30 días.

Impacto: Si bien la lectura pura del nodo demandó 6.666 ms, el tiempo final se disparó a 47.202 ms porque además hubo que agrupar los gastos, ordenar el resultado para quedarse con el Top 50 y coordinar el trabajo de los procesos paralelos.

Plan 1 Después (Con el índice idx_pedido_estado_fecha)

1. Bitmap Index Scan (sobre el índice)

¿Qué hizo la base de datos? Consultó el índice idx_pedido_estado_fecha, que guarda los pares (estado, fecha) ordenados. Gracias a ese orden, la base se posicionó directamente sobre el bloque de “terminados” dentro del rango de los últimos 30 días.

Resultado del nodo: No accedió a las filas todavía; solamente construyó un mapa de bits (listado de posiciones en disco) que señala las páginas donde están los registros que cumplen el filtro.

2. Bitmap Heap Scan on pedido (sobre la tabla)

¿Qué hizo la base de datos? Tomó ese mapa de bits y fue a la tabla pedido a recuperar únicamente los bloques de disco señalados, sin leer filas que no hicieran falta.

Tiempo real del nodo: Logró traer los registros exactos en apenas 0.183 ms.

¿Por qué el cambio dio tan buen resultado?

Eliminó la lectura completa: Se pasó de revisar toda la tabla a trabajar solo con el subconjunto de pedidos terminados y recientes.

Mejoró el acceso al disco: El Bitmap Scan organiza las lecturas físicas de forma ordenada, evitando movimientos erráticos entre páginas.

Efecto en cadena: Al reducirse fuertemente el volumen de datos que llega a las etapas posteriores (suma de gastos, ORDER BY y LIMIT 50), todo el proceso terminó casi instantáneamente, bajando el tiempo global a 6.768 ms.

Plan 2 Antes (Búsqueda tradicional con LIKE '%palabra%')

Nodo: Seq Scan on producto p

¿Qué hizo la base de datos? Al usarse un patrón con comodines del tipo %palabra%, PostgreSQL no pudo aprovechar ningún índice tradicional (como un B-Tree) y tuvo que hacer un escaneo secuencial sobre producto, recorriéndola fila por fila de punta a punta.

¿Por qué demoró tanto? Para cada fila de la tabla tuvo que comparar el contenido de descripcion contra la palabra buscada, carácter por carácter.

Impacto: El nodo llegó a consumir hasta 38.510 ms procesando texto de forma bruta, con lo cual el tiempo total quedó en 38.587 ms.

Plan 2 Después (Búsqueda de Texto Completo / Full-Text Search)

Al incorporar to_tsvector('spanish', descripcion) más un índice GIN (índice invertido generalizado), la consulta pasó de ser una búsqueda de caracteres a funcionar como un buscador de texto real.

1. Bitmap Index Scan (sobre el índice GIN)

¿Qué hizo la base de datos? El GIN se comporta como el índice temático de un libro: contiene un listado de lexemas (raíces de palabras en español) y para cada uno indica en qué filas aparece.

Resultado del nodo: PostgreSQL ubicó el término buscado en ese listado, detectó las coincidencias casi al instante y armó un mapa de bits con la ubicación física de esos productos.

2. Bitmap Heap Scan on producto

¿Qué hizo la base de datos? Con ese mapa de bits, fue a la tabla a recuperar solo las filas coincidentes, descartando por completo el resto de los productos.

Tiempo real del nodo: La operación tardó solo 0.018 ms porque leyó únicamente las páginas de memoria donde estaban los productos filtrados.

¿Por qué la mejora fue tan impactante?

De revisar todo a ir directo al dato: Se dejó de inspeccionar el 100% de las filas para acceder únicamente a los productos que contienen el término.

Normalización del lenguaje: to_tsvector('spanish', ...) trabaja con la raíz de cada palabra (maneja acentos, mayúsculas/minúsculas, plurales y conjugaciones), lo que hace que la búsqueda no solo sea muchísimo más rápida, sino también más exacta y útil para el usuario final.

4. Documentar los hallazgos en la tabla siguiente.

Afirmación de la IA	¿Correcta?	Corrección / evidencia del plan real
Descripción del plan y porque mejoró	Sí	La interpretación de la IA coincide con lo que refleja el plan de ejecución. No se detectaron errores ni confusiones
Descripción del plan viejo y porque mejoró	Sí	La descripción de la IA se ajusta fielmente al comportamiento observado en la base de datos. No hubo fallos o errores

Parte 4 — Consultas resumen y subconsultas bajo especificación precisa

Se trata de practicar el flujo de especificar con precisión —en lugar de pedir “hacé una consulta que...” de forma vaga— y de verificar que lo que la IA generó es equivalente a lo que se pidió, no solo que “corre sin error”.

1. Redactar una especificación precisa (spec) para al menos dos consultas sobre Food Store: una de resumen (agregación) y una con subconsulta. La especificación debe fijar explícitamente: tablas involucradas, filtro de borrado lógico, columnas de salida, orden y, si corresponde, criterio de corte (LIMIT, HAVING).
2. Pedirle a la IA que genere el SQL a partir de esa especificación, sin mostrarle una solución previa.
3. Escribir, de forma independiente, una segunda versión propia (o pedirle a la IA una alternativa con distinta estructura: subconsulta vs. join, por ejemplo) que responda la misma pregunta.
4. Verificar la equivalencia de resultados entre ambas versiones — por ejemplo, comparando el conteo de filas y una muestra de valores, o con EXCEPT entre ambos resultados.

```
-- Verificación de equivalencia entre dos consultas que deberían dar el mismo
resultado (consulta_A) EXCEPT (consulta_B);      -- filas en A que no están en B
(consulta_B) EXCEPT (consulta_A);      -- filas en B que no están en A
-- Ambas deben devolver 0 filas si son equivalentes
```

Ejemplo de especificación precisa para pedirle a la IA (a adaptar y completar por el estudiante, no copiar literal):

Spec de ejemplo

«Genera una consulta SQL sobre el esquema de Food Store que devuelva, para cada categoría vigente (eliminado = FALSE), el nombre de la categoría y la cantidad de productos vigentes que tiene, incluyendo las categorías sin productos vigentes con cantidad 0. Ordená de mayor a menor cantidad. No uses SELECT *.»

Consulta de Resumen (agregación).

Spec:

Trabajando sobre el esquema de Food Store (schema.sql), se requiere una consulta SQL que cumpla con los siguientes puntos:

- Listar para cada cliente activo (condición usuario.eliminado = FALSE) su id, su nombre y el número total de pedidos activos (pedido.eliminado = FALSE) que haya realizado.
- Los clientes que no tengan pedidos vigentes deben aparecer igualmente en el resultado, mostrando 0 en la columna de cantidad (no deben quedar excluidos).
- Los nombres de las columnas de salida deben ser exactamente: id, nombre, cantidad_pedidos.
- El conjunto de resultados debe ordenarse primero por cantidad_pedidos en orden descendente; en caso de empate, se ordenará por id en forma ascendente.
- No se permite el uso de SELECT * ni subconsultas: la solución debe implementarse mediante JOIN y GROUP BY.

Versión A:

```
SELECT
  c.id,
  c.nombres AS nombre,
  COUNT(co.id) AS cantidad_pedidos
FROM cliente c
LEFT JOIN compra co ON co.cliente_id = c.id AND co.eliminado = FALSE
WHERE c.eliminado = FALSE
GROUP BY c.id, c.nombres
ORDER BY cantidad_pedidos DESC, c.id ASC;
```

Versión B:

```
SELECT
  c.id,
  c.nombres AS nombre,
  (
    SELECT COUNT(co.id)
    FROM compra co
    WHERE co.cliente_id = c.id
```

```

        AND co.eliminado = FALSE
    ) AS cantidad_pedidos
FROM cliente c
WHERE c.eliminado = FALSE
ORDER BY cantidad_pedidos DESC, c.id ASC;

```

Consulta de Resumen (agregación).

Spec:

Dentro del esquema de Food Store, se requiere una consulta SQL que cumpla con lo siguiente:

- Mostrar todos los productos activos (producto.eliminado = FALSE) que estén asociados a categorías también activas (categoria.eliminado = FALSE).
- Incluir únicamente aquellos productos cuyo precio sea mayor al promedio de precios de todos los productos vigentes en la tienda.
- Las columnas de salida deben llamarse exactamente: id, nombre, precio.
- El cálculo del promedio debe realizarse obligatoriamente mediante una subconsulta escalar dentro de la cláusula WHERE.
- El resultado debe ordenarse primero por precio en orden descendente; en caso de igualdad, se ordenará por id en orden ascendente.
- No se permite el uso de SELECT *.

Versión A:

```

SELECT

    p.id,

    p.titulo AS nombre,

    p.valor AS precio

FROM articulo p

JOIN rubro c ON c.id = p.rubro_id

WHERE p.eliminado = FALSE

    AND c.eliminado = FALSE

    AND p.valor > (

        SELECT AVG(valor)

        FROM articulo

        WHERE eliminado = FALSE

    )

```

```
ORDER BY p.valor DESC, p.id ASC;
```

Versión B:

```
WITH promedio AS (
```

```
    SELECT AVG(valor) AS precio_promedio
```

```
    FROM articulo
```

```
    WHERE eliminado = FALSE
```

```
)
```

```
SELECT
```

```
    p.id,
```

```
    p.titulo AS nombre,
```

```
    p.valor AS precio
```

```
FROM articulo p
```

```
JOIN rubro c ON c.id = p.rubro_id
```

```
CROSS JOIN promedio pr
```

```
WHERE p.eliminado = FALSE
```

```
    AND c.eliminado = FALSE
```

```
    AND p.valor > pr.precio_promedio
```

```
ORDER BY p.valor DESC, p.id ASC;
```

Parte 5 — Competencia de optimización entre equipos

Cierre de la práctica: todos los equipos reciben la misma consulta lenta —fijada por la cátedra sobre la base masiva común— y compiten por lograr el mejor plan posible, con la IA como asistente de cada equipo.

- Consigna común: se entrega una única consulta lenta (por ejemplo, un listado de productos por categoría con filtro de precio y orden, sin índice) igual para todos los equipos.
- Cada equipo usa su asistente de IA para proponer reescrituras e índices, pero decide y aplica solo lo que puede justificar.
- Gana el equipo con el mejor tiempo de ejecución real (EXPLAIN ANALYZE), no el mejor costo estimado.
- Cada equipo documenta en su bitácora qué probó, qué descartó y por qué, incluso las propuestas de la IA que no funcionaron.

Registro de la competencia

Equipo	Estrategia aplicada	Tiempo antes (ms)	Tiempo después (ms)	Mejora (x)
Grupo E	Se implementó un índice B-Tree compuesto sobre (categoria_id, precio ASC), definido como parcial (WHERE eliminado = FALSE) y con columnas adicionales incluidas (nombre, stock) para habilitar un <i>Index Only Scan</i> y eliminar el nodo de ordenamiento.	29.540 ms	1.120 ms	26.3x

Declaración de Uso de IA (DUIA)

Obligatoria para toda la práctica, según la política de uso responsable de IA de la cátedra. Completar una fila por cada uso relevante de IA durante el trabajo.

Herramienta	Para qué se usó	Prompt / spec (resumen)	Se aceptó / se descartó — por qué
OpenCode	Analizar el plan de ejecución de la consulta de competencia y proponer el índice óptimo para suprimir	Proponé el mejor índice para la consulta de competencia (filtro por categoria_id 2, rango de precio, eliminado = FALSE, ordenado por precio). Busco eliminar el nodo Sort.	Aceptado: Índice compuesto B-Tree (categoria_id, precio) con INCLUDE (nombre, stock) y filtro WHERE eliminado = FALSE. Se aceptó porque elimina el

	cuellos de botella.		nodo Sort y permite Index Only Scan en RAM sin consultar el Heap.
Kiro	No fue utilizado		

Link al repositorio de Github: https://github.com/XavierPappalardo/UTN-TUPaD-TP_Base_de_Datos_Semana3_2PRO1

Entregables

- Script de carga masiva (leído, probado y ejecutado bajo el protocolo de seguridad).
- Tabla comparativa de la Parte 2, con capturas o texto de los planes antes/después.
- Tabla de lectura crítica de la Parte 3.
- Las dos consultas de la Parte 4 (spec + SQL generado + verificación de equivalencia).
- Registro de la competencia de la Parte 5.
- Declaración de Uso de IA (DUIA) completa.
- Defensa oral: cualquier script o índice de la entrega debe poder explicarse en el momento.

Criterios de evaluación

Dimensión	Qué se evalúa	Peso
Medición	EXPLAIN ANALYZE correcto y completo, antes y después, para cada cambio	30%
Criterio de aceptación	Las propuestas de la IA se validan, no se aplican a ciegas	25%
Lectura crítica	Se detectan imprecisiones reales en las explicaciones generadas por IA	20%
Equivalencia verificada	Las consultas alternativas se comprueban formalmente equivalentes	15%
Bitácora y defensa	DUIA completa y capacidad de explicar cada decisión	10%